

แบบทดสอบท้ายแล็บ

Custom ViewResolver ใน Spring Boot + Thymeleaf

วิชา CP353002 Principles of Software Design

ชื่อ-นามสกุล:พริมภัทร ภาวัฒน์คุณ รหัสนักศึกษา: 673380594-9

คำชี้แจง: ส่วนที่ 1 ปรนัย เลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว ส่วนที่ 2 อัตนัย ตอบเป็นข้อความอธิบาย ส่วนที่ 3 ปฏิบัติ
ให้ลงมือแก้ไขได้จริงในโปรเจกต์ของตัวเอง แล้ว capture หน้าจอผลลัพธ์แปะในกล่องที่เตรียมไว้

ส่วนที่ 1 — ปรนัย (Multiple Choice)

1. ในสถาปัตยกรรม MVC ส่วนใดที่ทำหน้าที่แปลง "ชื่อ view" ซึ่งตรงกรจะให้กลายเป็น path ของไฟล์ view จริง? (2 คะแนน)

- ก. Model
- ข. Controller
- ค. ViewResolver
- ง. DispatcherServlet

2. เมื่อ Controller เขียน return "home"; ค่า "home" ที่ถูกส่งกลับคืออะไร? (2 คะแนน)

- ก. Path ของไฟล์ HTML บนเครื่อง server
- ข. ชื่อ view ซึ่งตรงกรจะ (logical view name)
- ค. ชื่อ bean ของ ViewResolver
- ง. ชื่อ method ของ Controller

3. component ใดใน Spring MVC เป็นผู้เรียกใช้ ViewResolver โดยอัตโนมัติ หลังจาก Controller คืนค่าชื่อ view? (2 คะแนน)

- ก. HandlerMapping
- ข. DispatcherServlet
- ค. SpringTemplateEngine
- ง. HomeController

4. ใน ThymeleafConfig.java เมื่อบด resolver.setPrefix("classpath:/custom-templates/") ทำหน้าที่อะไร? (2 คะแนน)

- ก. กำหนด URL ที่ Controller จะรับ request
- ข. กำหนดโฟลเดอร์ต้นทางที่ ViewResolver จะค้นหาไฟล์ template

ค. กำหนดชื่อ database connection

ง. กำหนด port ที่แอปพลิเคชันรัน

5. ถ้ามีการลงทะเบียน **ViewResolver** มากกว่าหนึ่งตัวในบริบท (context) เดียวกัน Spring จะเลือกใช้ตัวไหนก่อน? (2 คะแนน)

ก. ตัวที่ประกาศเป็นตัวแรกในไฟล์เสมอ

ข. ตัวที่มีค่า order ต่ำกว่า (ถูกเรียกก่อน)

ค. สุ่มเลือก

ง. ตัวที่ตั้งชื่อ bean เรียงตามตัวอักษร

6. เพราะเหตุใด Controller ในตัวอย่างนี้จึงใช้ **@Controller** แทน **@RestController**? (2 คะแนน)

ก. เพราะ **@RestController** ใช้กับ Thymeleaf ไม่ได้เลย

ข. เพราะต้องการให้ค่าที่ return ถูกตีความเป็นชื่อ view ไม่ใช่ response body โดยตรง

ค. เพราะ **@Controller** ทำงานเร็วกว่า

ง. ไม่มีความแตกต่างกันเลย

ส่วนที่ 2 — อรรถนัย (Short Answer / Essay)

1. ใครเป็นผู้เรียกใช้ Custom ViewResolver และเรียกเมื่อไร? จงอธิบายเป็นลำดับขั้นตอน ตั้งแต่ Browser ส่ง request จนได้ HTML response กลับ (5 คะแนน)

ตอบ ผู้เรียกคือ **DispatcherServlet** เรียกหลังจาก Controller return ชื่อ view เสร็จแล้ว ลำดับขั้นตอน

1. Browser ส่ง GET request ไปที่ /
2. DispatcherServlet รับ request แล้วส่งต่อให้ HomeController
3. HomeController.home() ทำงาน ใส่ค่าใน Model แล้ว return string "home"
4. DispatcherServlet เอาชื่อ "home" ไปส่งให้ ViewResolver (ที่เราลงทะเบียนใน ThymeleafConfig)
5. ViewResolver แปลง "home" → classpath:/custom-templates/home.html โดยใช้ prefix + suffix
6. Thymeleaf template engine render ไฟล์ HTML นั้น แทนค่า \${message} ด้วยข้อมูลจาก Model
7. DispatcherServlet ส่ง HTML ที่ render เสร็จกลับไปเป็น response ให้ Browser

2. จงอธิบายว่าเหตุใดการแยก ViewResolver ออกจาก Controller จึงสอดคล้องกับหลักการ separation of concerns และ dependency inversion ใน Principles of Software Design (5 คะแนน)

ตอบ

- **Separation of concerns:** Controller มีหน้าที่รับ request และตัดสินใจ logic เท่านั้น ไม่ควรรู้จัก path ไฟล์จริงบน disk ส่วน ViewResolver แยกไปรับผิดชอบเฉพาะการแปลง ชื่อ view เป็น ไฟล์จริง คนละหน้าที่ ไม่ปนกัน
 - **Dependency inversion:** Controller ไม่ได้ผูกติด (depend) กับ path หรือ template engine ที่ใช้จริงโดยตรง แต่พึ่งพาแค่ ชื่อ view ซึ่งเป็นนามธรรมทำให้เปลี่ยนโพลเดอร์ปลายทาง เปลี่ยน naming convention หรือแม้แต่สลับ template engine ได้ โดยไม่ต้องแก้โค้ด Controller เลย
-

3. หากต้องการเพิ่ม view ใหม่ชื่อ "about" โดยยังใช้ custom ViewResolver ตัวเดิม ต้องสร้างหรือแก้ไขไฟล์ใดบ้าง?

ระบุชื่อไฟล์และ path ให้ครบถ้วน (3 คะแนน)

ตอบ ไม่ต้องแก้ ThymeleafConfig.java เลย (ตัว ViewResolver ใช้อยู่กับทุก view ที่ชื่อไหนก็ได้อยู่แล้ว) แค่เพิ่ม 2 ไฟล์:

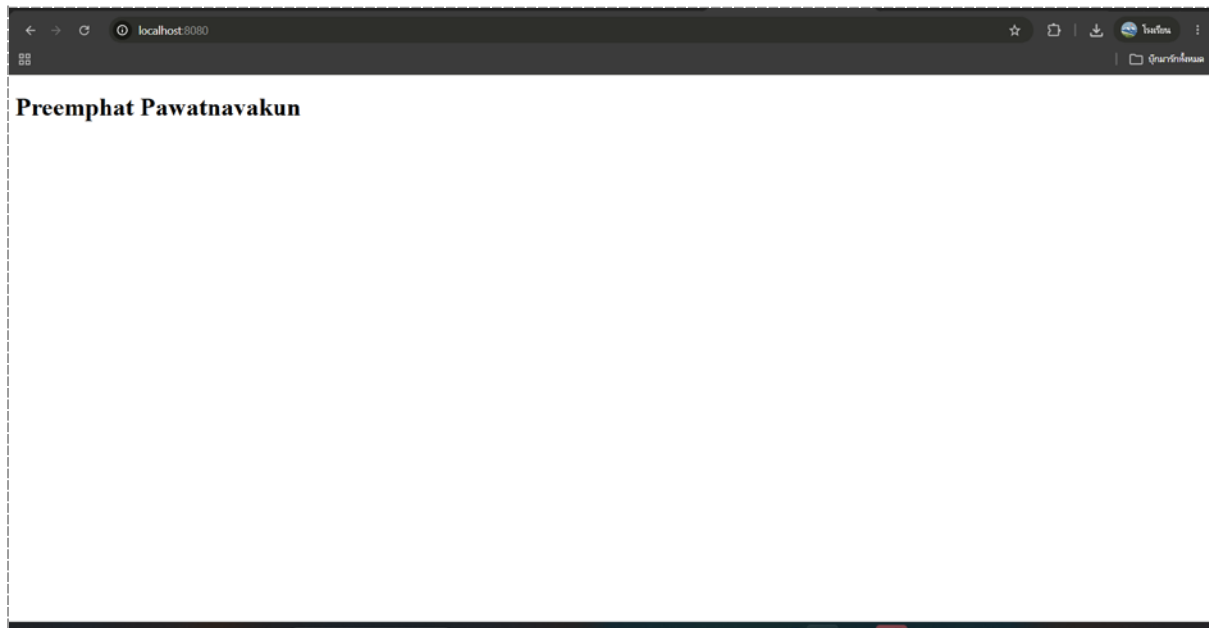
สร้างไฟล์ใหม่: src/main/resources/custom-templates/about.html

แก้ไฟล์เดิม: src/main/java/com/example/spring_thymeleaf_demo/controller/HomeController.java — เพิ่ม method ใหม่
ข้างใน class เดิม

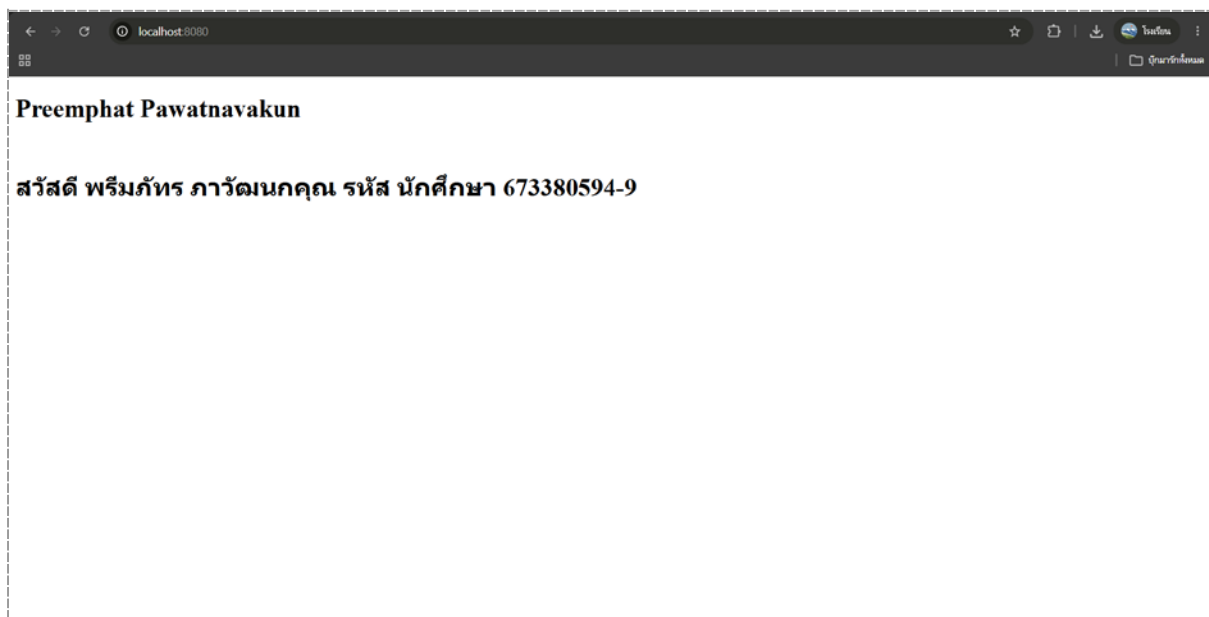
ส่วนที่ 3 — ปฏิบัติ (แก้โค้ดจริง + แบนภาพหน้าจอ)

ทำตามคำสั่งแต่ละข้อในโปรเจกต์ของตัวเอง รันด้วย `mvn spring-boot:run` แล้ว *capture* หน้าจอผลลัพธ์แปะในกล่องเส้นประด้านล่างแต่ละข้อ

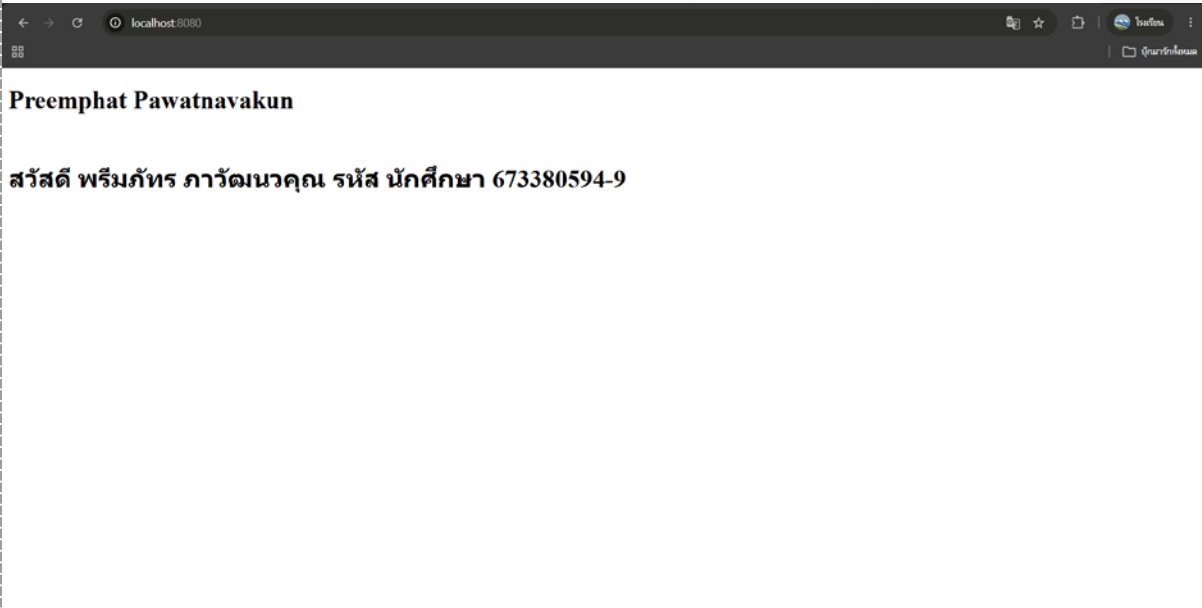
1. แก้ไขค่าตัวแปร `message` ใน `HomeController.java` ให้แสดงชื่อ-นามสกุลของตัวเองแทนข้อความเดิม แล้ว *capture* หน้าเว็บ `http://localhost:8080/` ที่แสดงชื่อของตัวเอง (3 คะแนน)



2. เพิ่มตัวแปรใหม่ชื่อ `studentId` ส่งจาก Controller ไปแสดงต่อจากชื่อในหน้า `home.html` (เช่น "สวัสดี ชื่อ (รหัส XXX)") แล้ว *capture* หน้าเว็บที่แสดงผลลัพธ์ (3 คะแนน)



3. เปลี่ยนค่า resolver.setPrefix() ใน ThymeleafConfig.java ให้ใช้ไปโฟลเดอร์ใหม่ชื่อ my-templates/ แทน custom-templates/ (ต้องย้ายไฟล์ home.html ไปด้วย) แล้ว capture ทั้งโค้ดที่แก้ และหน้าเว็บที่ยังทำงานปกติ (4 คะแนน)



The screenshot shows a web browser at localhost:8080 displaying a page with the following content:

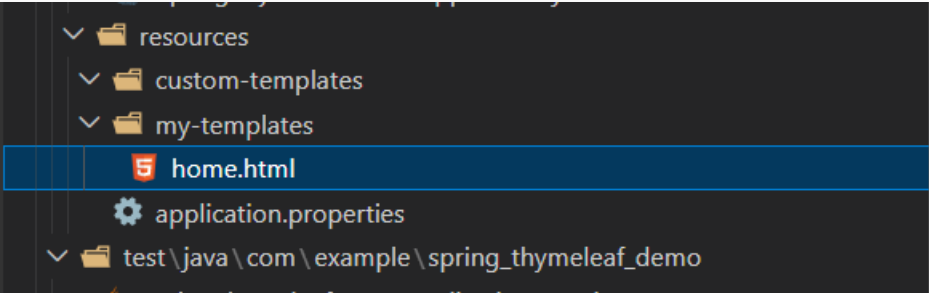
Preemphat Pawatnavakun

สวัสดี พร้มภัทร ภาว์ฉนวนคุณ รหัส นักศึกษา 673380594-9



The screenshot shows the ThymeleafConfig.java file in an IDE. The code is as follows:

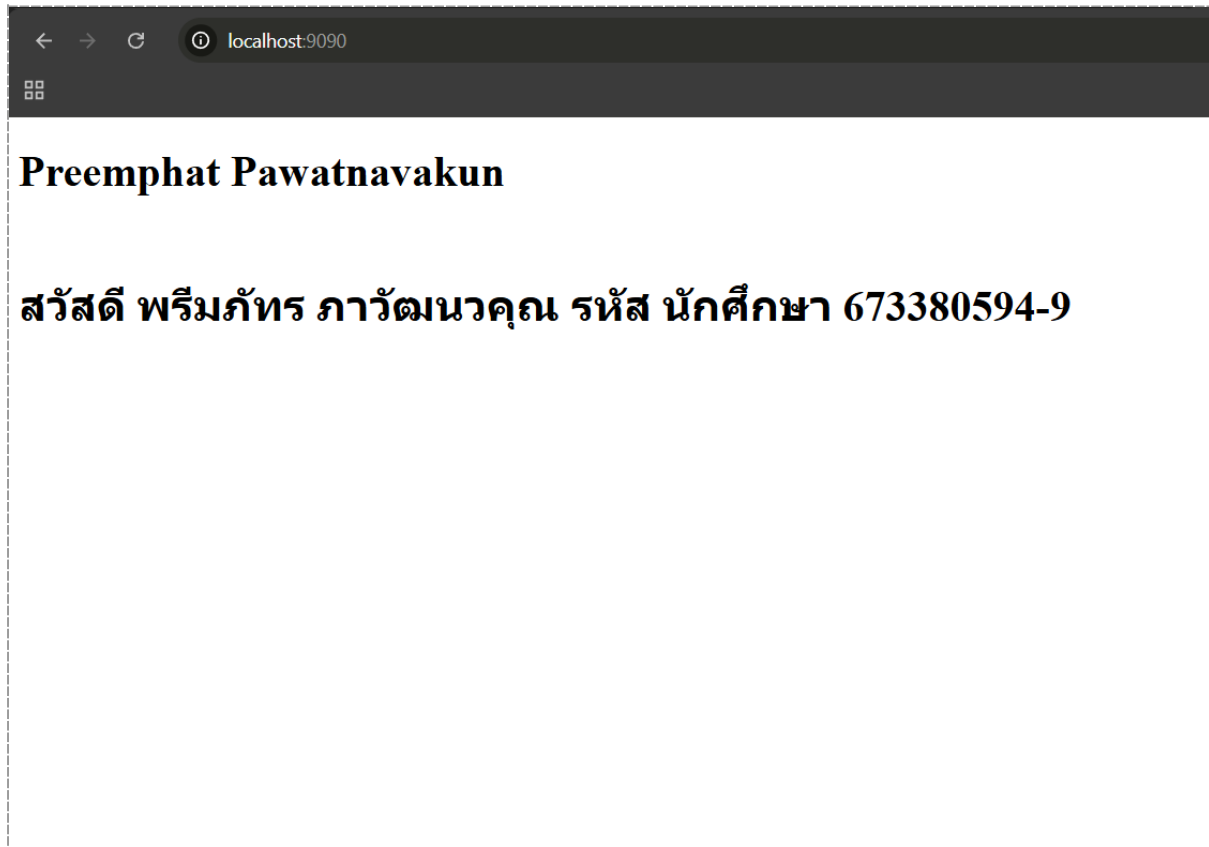
```
1 package com.example.spring_thymeleaf_demo.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.thymeleaf.spring6.SpringTemplateEngine;
6 import org.thymeleaf.spring6.templateresolver.SpringResourceTemplateResolver;
7 import org.thymeleaf.spring6.view.ThymeleafViewResolver;
8
9 @Configuration
10 public class ThymeleafConfig {
11     @Bean
12     public SpringResourceTemplateResolver templateResolver() {
13         SpringResourceTemplateResolver resolver = new SpringResourceTemplateResolver();
14         resolver.setPrefix("classpath:/my-templates/");
15         resolver.setSuffix(".html");
16         resolver.setTemplateMode("HTML");
17         resolver.setCharacterEncoding("UTF-8");
18         resolver.setCacheable(false);
19         return resolver;
20     }
21
22     @Bean
23     public SpringTemplateEngine templateEngine(SpringResourceTemplateResolver templateResolver) {
24         SpringTemplateEngine engine = new SpringTemplateEngine();
25         engine.setTemplateResolver(templateResolver);
26         return engine;
27     }
28 }
```

The screenshot shows the IDE's file explorer with the following structure:

- resources
 - custom-templates
 - my-templates
 - home.html
- application.properties
- test\java\com\example\spring_thymeleaf_demo

4. เปลี่ยน server.port ใน application.properties เป็น 9090 แล้ว capture หน้าเว็บที่ URL เปลี่ยนเป็น `http://localhost:9090/` (3 คะแนน)



5. เพิ่ม view ใหม่ชื่อ "about" (เพิ่ม `@GetMapping("/about")` และไฟล์ `about.html`) ให้แสดงข้อความแนะนำตัวสั้นๆ แล้ว capture หน้าเว็บ `/about` ที่แสดงผลลัพธ์ (4 คะแนน)

